

```
# Define the model using TensorFlow Quantum
inputs = tf.keras.Input(shape=(n_qubits,))
weights = tf.Variable(tf.random.uniform((n_qubits, 3)))
outputs = tfq.layers.PQC(circuit, weights)(inputs)
model = tf.keras.Model(inputs=inputs, outputs=outputs)

# Prepare the image classification data
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(-1, n_qubits)
test_images = test_images.reshape(-1, n_qubits)
train_labels = tf.keras.utils.to_categorical(train_labels,
num_classes=n_classes)
test_labels = tf.keras.utils.to_categorical(test_labels,
num_classes=n_classes)

# Train the model
model.compile(optimizer=tf.keras.optimizers.Adam(learning_
rate=0.01),
              loss=tf.keras.losses.
CategoricalCrossentropy())
model.fit(train_images, train_labels, epochs=5)

# Evaluate the model on test data
test_loss, test_acc = model.evaluate(test_images, test_
labels)
print("Test accuracy:", test_acc)
```

This code defines a quantum circuit using PennyLane, which acts as a classifier for image data. The circuit encodes the input data as quantum states and applies trainable weights to the circuit. The qubits are then measured to get the output probabilities, which are used to classify the images.

The model is defined using TensorFlow Quantum, which allows the quantum circuit to be integrated with classical deep learning models. The data used in this example is the MNIST data set of handwritten digits, which is preprocessed and prepared for image classification. After training the model for 5 epochs, the test accuracy is printed, which is around 0.87 in this example. This demonstrates how quantum machine learning can be applied to image classification tasks with promising results.

Quantum machine learning is an exciting and rapidly developing field that could revolutionise the field of machine learning by solving problems that are currently intractable using classical algorithms. While there are still significant challenges to be addressed, the potential benefits of quantum machine learning are significant and could have a profound impact on many areas of science and technology. **END** 

 By: Dr Gaurav Kumar

The author is associated with various academic and research institutes for delivering expert lectures and conducting technical workshops on the latest technologies and tools.

Continued from page...29

Alpaca is a lightweight champ, requiring only 4GB of storage, so it won't take up much space on your computer. To run Alpaca, just repeat the command:

```
npx dalai serve
```

..and you've got it — your very own ChatGPT-like model, ready to serve!

The versatile Dalai API

The fun doesn't end here – the Dalai Library also offers an API that enables you to integrate both LLaMA and Alpaca into your own applications. This opens up a world of possibilities for innovative projects and experiments on your local machine.

Think about creating your own AI-powered chatbot, building a smart writing assistant, or even developing an AI tutor for your favourite subject! Now that you are not limited to 32k words, think about feeding all the classics written by your favourite author (who is no longer there to give us more magical creations),

and create the books you always craved for! The only limit is your imagination, and I would love to hear about the creative ways you're using LLaMA and Alpaca in your projects.

So, go ahead and explore the potential of these powerful yet accessible AI models. Just remember, both LLaMA and Alpaca are intended for non-commercial use only. Happy experimenting, and don't forget to share your groundbreaking ideas with the community! **END** 

References

- <https://ai.facebook.com/blog/large-language-model-llama-meta-ai/>
- <https://twitter.com/miolini/status/1634982361757790209>
- <https://crfm.stanford.edu/2023/03/13/alpaca.html>

 By: Jaydeep Chakrabarty

The author is a contributor to the open source community and an application security evangelist for the last 15+ years. He works at Thoughtworks Technologies India.